Backend + Agent Design Flow (post-ADR-0011 target)

Purpose: Reference diagrams for the backend interface v2 + agent core design decided during the ADR 0011 walkthrough (2026-08-21).

Scope note: This document shows the **target design** after ADR 0011 + ADR 0012 ship. The current Phase 1 codebase reflects an earlier state (OpenAIBackend only, no registry, simpler warmup semantics). Diagrams here are the shape you're building TO, not the shape currently deployed.

Related:

- ADR 0011 (backend interface v2 — to write)
- ADR 0012 (model registry — to write, co-designed with 0011)
- METHODOLOGY.md
- ARCHITECTURE.md

---

1. Component composition

The system's structural layering. Shows what holds what.

```
flowchart TB
    subgraph FastAPI["FastAPI Application"]
        API["/invoke endpoint"]
        HEALTH["/health endpoint"]
        LIFESPAN["lifespan()<br/>startup + shutdown"]
    end

    subgraph AGENT_LAYER["Agent Layer (per-request)"]
        AGENT["Agent<br/>handle_request()"]
        VAL["Validator"]
        DISP["FakeAdapterDispatcher"]
        TEL["TelemetryLogger<br/>+ CostAccumulator"]
    end

    subgraph BACKEND_LAYER["Backend Layer (singleton)"]
        BASE["InferenceBackend ABC<br/>plan_task() / interpret_results()<br/>_call_llm() with retry"]
        OAI["OpenAIBackend"]
        MLX["MLXBackend"]
        FAKE["FakeBackend<br/>tests only"]
    end
```

```
    subgraph SUPPORT["Support"]
        REG[("models.json<br/>registry manifest")]
        WARM[("warmup_scenario.json")]
        ENV[("env
vars<br/>BAYMAX_BACKEND<br/>BAYMAX_MODEL_ID<br/>BAYMAX_RETRY_MAX_ATTEMPTS<br/>OPENAI_API_KEY")]
        ADAPTERS[("Ronin's fake adapters<br/>calendar / notion / gmail / clipboard")]
    end

    LIFESPAN -->|instantiates| OAI
    LIFESPAN -->|instantiates| MLX
    LIFESPAN -->|owns singleton| BASE
    LIFESPAN -->|reads at construction| ENV
    LIFESPAN -->|reads at construction| REG
    LIFESPAN -->|reads warmup prompt| WARM

    API -->|per request| AGENT
    AGENT --> VAL
    AGENT --> DISP
    AGENT --> TEL
    AGENT -->|calls| BASE

    OAI -.-> BASE
    MLX -.-> BASE
    FAKE -.-> BASE

    DISP --> ADAPTERS

    HEALTH -->|reads warmup state| LIFESPAN

    classDef ext fill:#e8f0ff,stroke:#4a6faf,color:#000
    classDef support fill:#fff4e0,stroke:#c48800,color:#000
    class FastAPI ext
    class SUPPORT support
```

Key composition rules (from A/B/G locks):

- Agent HOLDS backend (dependency injection); Agent does not KNOW backend type
- Backend HOLDS registry entry (registry-driven metadata per G1)
- FastAPI lifespan OWNS the singleton backend instance
- Everything reads env vars in `__init__` (per B4); no global env access elsewhere

---

2. Startup lifecycle

The sequence from `uvicorn baymax.service.api:app` to `/health` returning 200.

```
flowchart TD
    START([uvicorn starts])
    IMPORT[Import module<br/>side-effect free]
    LIFESPAN_ENTER[lifespan begins]

    ENV_READ[Read BAYMAX_BACKEND<br/>+ BAYMAX_MODEL_ID env]
    ENV_VALID{Env valid?}
    ENV_FAIL[Raise ValueError<br/>uvicorn exits]

    FACTORY[BACKENDS dict lookup<br/>OpenAIBackend or MLXBackend]

    REG_READ[Load models.json<br/>fetch model_id entry]
    REG_VALID{Entry valid?<br/>required fields present?}
    REG_FAIL[Raise ModelNotFound / IncompleteRegistryEntry<br/>uvicorn exits]

    BACKEND_INIT[Backend __init__<br/>fetches OPENAI_API_KEY or MLX_MODEL_PATH<br/>raises loud on missing]
    BACKEND_FAIL{Init succeeded?}
    BACKEND_FAIL_EXIT[Raise ConfigError<br/>uvicorn exits]

    WARM_LOAD[Read warmup_scenario.json]
    WARM_RUN[Execute dummy scenario<br/>full backend call incl. tool-call generation]
    WARM_TIMEOUT{Completed within 120s?}
    WARM_TIMEOUT_FAIL[Raise WarmupTimeout<br/>uvicorn exits]
    WARM_ERR{Warmup succeeded?}
    WARM_ERR_FAIL[Raise WarmupFailure with stage+cause+hint<br/>uvicorn exits]

    READY[Set warmed_up = True<br/>Log 'backend ready' event with is_warmup=true tag]
    YIELD[lifespan yields<br/>server accepts requests]

    HEALTH_200[/health returns 200<br/>with backend metadata JSON]

    START --> IMPORT
    IMPORT --> LIFESPAN_ENTER
    LIFESPAN_ENTER --> ENV_READ
    ENV_READ --> ENV_VALID
    ENV_VALID -- no --> ENV_FAIL
    ENV_VALID -- yes --> FACTORY
    FACTORY --> REG_READ
    REG_READ --> REG_VALID
    REG_VALID -- no --> REG_FAIL
    REG_VALID -- yes --> BACKEND_INIT
    BACKEND_INIT --> BACKEND_FAIL
    BACKEND_FAIL -- no --> BACKEND_FAIL_EXIT
    BACKEND_FAIL -- yes --> WARM_LOAD
    WARM_LOAD --> WARM_RUN
    WARM_RUN --> WARM_TIMEOUT
    WARM_TIMEOUT -- no --> WARM_TIMEOUT_FAIL
```

```
    WARM_TIMEOUT -- yes --> WARM_ERR
    WARM_ERR -- no --> WARM_ERR_FAIL
    WARM_ERR -- yes --> READY
    READY --> YIELD
    YIELD --> HEALTH_200

    classDef fail fill:#ffcdd2,stroke:#c62828,color:#000
    classDef success fill:#c8e6c9,stroke:#2e7d32,color:#000
    class ENV_FAIL,REG_FAIL,BACKEND_FAIL_EXIT,WARM_TIMEOUT_FAIL,WARM_ERR_FAIL fail
    class READY,HEALTH_200 success
```

Failure semantics (per B5, C3, C4):

- All startup errors = process exit before `lifespan` yields
- `/health` never reaches ready state; uvicorn logs the error and dies
- No retry on warmup failures (deterministic errors, fail-loud policy)

Health response body (per C2):

```
// During warmup (503):
{"ready": false, "reason": "warming_up"}

// After warmup (200):
{"ready": true, "backend": "mlx", "model_id": "qwen-1.5b-baymax-v1"}
```

---

3. Request flow (happy path)

Single request from user through agent to response.

```
sequenceDiagram
    participant U as Client
    participant API as /invoke endpoint
    participant AG as Agent.handle_request
    participant V as Validator
    participant B as Backend
    participant D as Dispatcher
    participant T as TelemetryLogger

    U->>API: POST /invoke {request}
    API->>AG: handle_request(payload)

    Note over AG,T: telemetry BEGINS before validation

    AG->>T: log request_received
    AG->>V: validate_task_file(task)
    V-->>AG: valid TaskFile
```

```
    AG->>B: plan_task(task) [wrapped in _call_llm]
    Note over B: retry loop D1-D4 if needed
    B-->>AG: PlanResult(steps, tokens, cost, warnings)

    loop each step in plan
        AG->>V: validate_tool_call(step)
        V-->>AG: valid
        AG->>D: dispatch(step)
        D-->>AG: ToolCallResult
        AG->>T: log tool_call event
    end

    AG->>B: interpret_results(task, results)
    Note over B: retry loop D1-D4 if needed
    B-->>AG: interpretation

    AG->>AG: task.to_agent_response()
    AG->>T: log request_completed
    AG-->>API: AgentResponse
    API-->>U: 200 OK {AgentResponse}
```

Order guarantees (from ADR 0001, 0002):

- Telemetry always fires BEFORE validation (per Phase 1 lock — captures failed-task_file-construction events)
- Validator runs on both the initial TaskFile and each ToolCall step before dispatch
- Agent uses meta-tools (`request_clarification`, `refuse_request`) as normal tool calls — dispatched but with no side effects, sets response_type accordingly

Two distinct retry loops (do not confuse):

- **Backend retry** (§4 below): inside `_call_llm`, retries transient LLM-API failures (HTTP 408/429/500-504, connection errors). Hardcoded N=3 per ADR 0011 §D. Never retries MLX.
- **Tool-call retry (ReAct-style recovery)**: inside the agent loop, retries individual TOOL dispatches that fail per `AgentConfig.max_retries_per_step` (ADR 0001). Runs at a higher layer than backend retry — a single tool-call attempt may internally consume up to N=3 backend attempts, so worst-case a `max_retries_per_step=2` scenario can produce 2×3=6 LLM calls if planning also gets retried.

---

4. Retry loop internals (inside `_call_llm`)

The retry logic that lives on the base class per A4, implementing D1-D5.

```
flowchart TD
    ENTER[_call_llm called]
    INIT[attempt = 1<br/>max_attempts = 3 hardcoded]
    LOG_ATTEMPT[Log 'backend_attempt' event<br/>attempt=N is_warmup=false]
```

```
    CALL[Execute HTTP or MLX call]
    RESULT{Result type?}

    SUCCESS[Log 'backend_call_complete' aggregate event<br/>total_attempts=N final_status=success]
    RETURN[Return result to caller]

    ERR_CLASSIFY{Classify error}
    NON_RETRIABLE[HTTP 400/401/403/404/422<br/>OR any MLX error]
    RETRIABLE[HTTP 408/429/500-504<br/>OR connection error<br/>on API backend only]

    RETRIABLE_CHECK{attempt < max_attempts?}
    BACKOFF[delay = base * factor^attempt<br/>>= 1s * 2^attempt<br/>apply ±25% jitter]
    LOG_RETRY[Log 'backend_retry' event<br/>attempt=N delay_before_ms=X cause=Y]
    SLEEP[await asyncio.sleep delay]
    INCREMENT[attempt += 1]

    EXHAUSTED[All attempts exhausted]
    LOG_FINAL_FAIL[Log 'backend_call_complete' aggregate<br/>total_attempts=N
final_status=failed<br/>with full attempt history]
    RAISE_BE[Raise BackendError<br/>kind=retry_exhausted OR
non_retriable<br/>underlying=orig_exception<br/>attempts_history=list of attempts]

    ENTER --> INIT
    INIT --> LOG_ATTEMPT
    LOG_ATTEMPT --> CALL
    CALL --> RESULT

    RESULT -- ok --> SUCCESS
    SUCCESS --> RETURN

    RESULT -- error --> ERR_CLASSIFY
    ERR_CLASSIFY --> NON_RETRIABLE
    ERR_CLASSIFY --> RETRIABLE

    NON_RETRIABLE --> LOG_FINAL_FAIL
    LOG_FINAL_FAIL --> RAISE_BE

    RETRIABLE --> RETRIABLE_CHECK
    RETRIABLE_CHECK -- no more attempts --> EXHAUSTED
    EXHAUSTED --> LOG_FINAL_FAIL

    RETRIABLE_CHECK -- retry --> BACKOFF
    BACKOFF --> LOG_RETRY
    LOG_RETRY --> SLEEP
    SLEEP --> INCREMENT
    INCREMENT --> LOG_ATTEMPT
```

```
    classDef fail fill:#ffcdd2,stroke:#c62828,color:#000
    classDef success fill:#c8e6c9,stroke:#2e7d32,color:#000
    class RAISE_BE,LOG_FINAL_FAIL fail
    class SUCCESS,RETURN success
```

Backoff formula (per D2, `attempt` is 1-indexed and identifies the JUST-FAILED attempt):

```
delay = base_delay * (factor ** (attempt - 1))
      = 1s * (2 ** (attempt - 1))
      → 1s wait before retry 2, 2s wait before retry 3, 4s wait before retry 4
delay_with_jitter = delay * uniform(0.75, 1.25)
```

N = 3 hardcoded (2026-08-24 revision): earlier design had N configurable via env var + eval sweep at multiple values; trimmed as publication-review overhead per stepping-stone reframe.

---

5. Error surfacing → HTTP 500

How a `BackendError` (from retry exhaustion or non-retriable error) reaches the client. **Revised 2026-08-26**: prior version had this returning HTTP 200 with `error_type` on `AgentResponse` — that broke `AgentResponse`'s boundary contract with Ronin's scorer. Corrected below.

```
flowchart LR
    subgraph BACKEND
        BE[Backend raises BackendError<br/>kind=retry_exhausted OR non_retriable<br/>underlying=...
attempts=...]
    end

    subgraph AGENT
        BUBBLE[Agent does NOT catch<br/>internal
TaskFile.completion_status=FAILED<br/>TaskFile.error_type=BACKEND_ERROR<br/>full attempts logged to
action_log for forensics]
    end

    subgraph FASTAPI
        HANDLER[app.exception_handler BackendError<br/>registered in lifespan]
        BUILD500[Build 500 body:<br/>error backend_error<br/>kind retry_exhausted or
non_retriable<br/>backend provider<br/>message brief<br/>attempts count]
    end

    subgraph CLIENT
        RESP[HTTP 500 + JSON error body<br/>AgentResponse boundary intact / not returned]
    end

    BE --> BUBBLE
    BUBBLE --> HANDLER
    HANDLER --> BUILD500
```

```
    BUILD500 --> RESP

    classDef fail fill:#ffcdd2,stroke:#c62828,color:#000
    class BE,BUBBLE fail
```

Key semantic (per E2, revised): backend failure is infrastructure-layer, not an agent decision. HTTP status codes exist for exactly this distinction:

- **200 + AgentResponse** = agent made a decision (execute, refuse, clarify)
- **500 + error body** = infrastructure failed (backend down, retries exhausted, warmup incomplete)

`AgentResponse` boundary shape stays `{tool_calls, message}` unchanged — Ronin's scorer contract intact.

500 body shape:

```
{"error": "backend_error", "kind": "retry_exhausted", "backend": "openai",
 "message": "openai backend failed after 3 attempt(s): http_429",
 "attempts": 3}
```

No stack traces or underlying reprs in the body — those live in the trace file per ADR 0010 redaction discipline.

Ronin coordination: eval runner needs try/except around the httpx call and a "backend_error" bucket in scoring categorization. Flag in next sync.

---

6. Registry lookup path (per G1)

How a backend obtains its model metadata at construction.

```
flowchart LR
    START[Backend __init__ called<br/>model_id='qwen-1.5b-baymax-v1']

    LOAD[Open models/models.json]
    PARSE[Parse manifest]
    LOOKUP[Find entry where entry.id == model_id]
    FOUND{Entry found?}
    NOT_FOUND[Raise ModelNotFound<br/>uvicorn exits at startup]

    VALIDATE_REQ{All required fields present?<br/>id base adapter_path trained_at
training_config_hash}
    INVALID[Raise IncompleteEntry<br/>uvicorn exits]

    EXTRACT[Extract metadata:
<br/>base_model<br/>adapter_path<br/>max_context<br/>cost_model<br/>tokenizer_path]

    STORE[Store on self as instance attributes<br/>for lifetime of process]

    START --> LOAD
```

```
    LOAD --> PARSE
    PARSE --> LOOKUP
    LOOKUP --> FOUND
    FOUND -- no --> NOT_FOUND
    FOUND -- yes --> VALIDATE_REQ
    VALIDATE_REQ -- no --> INVALID
    VALIDATE_REQ -- yes --> EXTRACT
    EXTRACT --> STORE

    classDef fail fill:#ffcdd2,stroke:#c62828,color:#000
    class NOT_FOUND,INVALID fail
```

Consequence of registry-driven (per G1): adding a new fine-tuned model is a REGISTRY UPDATE, not a backend code change. Ronin appends to `models.json` + drops adapter weights → backend picks it up on next startup with `BAYMAX_MODEL_ID` pointing at the new id.

Immutability guarantee (per RONIN_SYNC B3): a given `model_id` NEVER has its metadata rewritten. Every retrain = new id (`v1` → `v2` → `v3`). Any prior eval result labeled with the old id stays reproducible.

---

7. Testing surface (per F)

Where fakes and mocks live and how they interact.

```
flowchart TB
    subgraph PRODUCTION
        PROD_APP[baymax.service.api:app<br/>with real backend from BACKENDS dict]
        PROD_BE[OpenAIBackend or MLXBackend]
    end

    subgraph UNIT_TESTS[Unit Tests]
        UT_AGENT[Test Agent.handle_request<br/>with FakeBackend]
        UT_RETRY[Test retry loop<br/>with FakeBackend + patched asyncio.sleep]
        UT_WARMUP[Test warmup gate<br/>with FakeBackend + warmup_delay_s]
        UT_ADAPTER[Test output adapter<br/>with hardcoded inputs]
        FAKE[FakeBackend<br/>tests/fakes/backend.py<br/>canned responses<br/>injectable delays]
    end

    subgraph INT_TESTS[Integration Tests]
        IT_OAI[Test real OpenAI round-trip<br/>opt-in via pytest.mark.integration]
        IT_MLX[Test real MLX round-trip<br/>Marv-local only, not in CI]
    end

    UT_AGENT --> FAKE
    UT_RETRY --> FAKE
    UT_WARMUP --> FAKE

    PROD_APP --> PROD_BE
```

```
    IT_OAI -.->|hits real API| PROD_BE
    IT_MLX -.->|hits local MLX| PROD_BE

    classDef test fill:#f0f4c3,stroke:#827717,color:#000
    class UNIT_TESTS,INT_TESTS test
```

Test tiers:

Tier	Runs when	Env
Unit tests (FakeBackend)	Every commit / PR	No env needed
Integration OpenAI	`[test-integration]` PR tag OR manual	`OPENAI_API_KEY` set
Integration MLX	Marv-local only, before pushing MLX changes	Mac with weights downloaded

Not scheduled: no nightly CI runs. Ahmed's eval runs naturally catch drift.

---

8. Cross-diagram invariants

Rules that hold across all flows:

1. **Backend uniformity** — Agent code has ZERO backend-conditional logic. Every backend implements identical semantics. Format differences absorbed inside backend implementations.
2. **Backend metadata exposed but not consumed by agent** — `backend.max_context`, `backend.cost_model` accessible for telemetry/debugging, NOT for agent branching.
3. **Registry is source of truth for model facts** — no duplication in backend code.
4. **Immutable model IDs** — `v1` weights + metadata never change. Retrain → new ID.
5. **Fail-loud on deterministic errors** — MLX errors, config errors, warmup errors all exit the process rather than retry.
6. **Retry only on network-transient errors** — API backends only. All retry attempts logged per-event AND aggregated on completion (dual telemetry per D5).
7. **Telemetry BEFORE validation** — request events fire before validator runs, so failed-task_file-construction events are captured.
8. **No agent-level backend awareness** — Agent gets a backend via dependency injection, treats it as opaque.

---

9. What's NOT in this design (still deferred)

Called out so they're not confused with the current design:

- **Streaming inference** (A5) — deferred to v2; new `plan_task_stream()` method when added, does not break v1 callers

- **State store beyond in-memory** — ADR 0007 downgraded to v2 target; no SQLite in v1
- **Multi-turn dialog state** — v2/v3 per DEFERRED.md
- **Real tool adapters** (Notion/Gmail/Calendar/Clipboard) — v3+; v1 uses Ronin's fake adapters
- **LLM-as-judge scoring** — ADR 0008 scaffold, implementation deferred
- **Third + fourth backend classes** (Anthropic, other MLX variants) — factory dict makes trivial to add; not v1 scope
- **Backend capability declaration via `.capabilities()`** — considered and rejected in favor of interface uniformity (§8 invariant #1). If a 4th+ backend surfaces genuine capability divergence, revisit.

Written: 2026-08-21 as reference design artifact from ADR 0011 walkthrough.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.